

Module 1

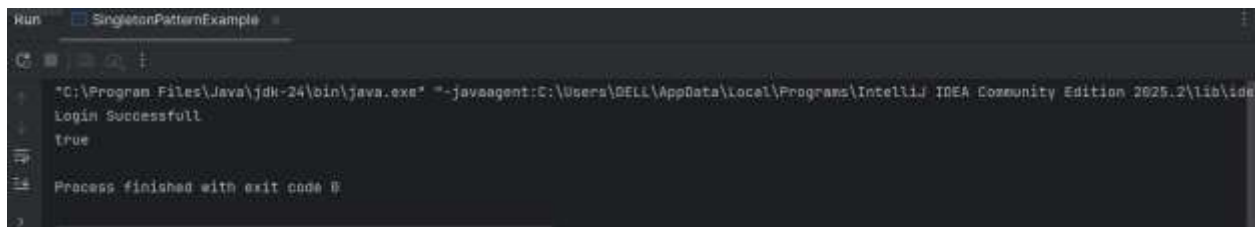
Exercise 1: Implementing the Singleton Pattern

```
public class SingletonPatternExample{
    public static void main(String[] args){
        Logger login1 = Logger.getInstance();
        Logger login2 = Logger.getInstance();

        System.out.println(login1 == login2);
    }
}

class Logger{
    private static Logger login;
    private Logger(){
        System.out.println("Login Successfull");
    }

    public static Logger getInstance(){
        if(login == null){
            synchronized(Logger.class){
                if(login == null){
                    login = new Logger();
                }
            }
        }
        return login;
    }
}
```



```
run SingletonPatternExample
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\DELL\AppData\Local\Programs\IntelliJ IDEA Community Edition 2025.2\lib\idea
Login Successfull
true
Process finished with exit code 0
```

In this program, I implemented the Singleton Design Pattern using a Logger class. The main purpose of Singleton is to make sure that only one object of a class is created and used throughout the program. To achieve this, I made the constructor private so that no other class can create objects directly. I used a static variable to store the single object and a `getInstance()` method to return that object whenever needed. I also used synchronized double-checking to make the code thread-safe, so even if multiple threads access it at the same time, only one object will be created. When I tested the program, both `login1` and `login2` referred to the same Logger object, proving that the Singleton pattern was implemented successfully.

Exercise 2: Implementing the Factory Method Pattern

```
interface Document {
    void open();
}

class WordDocument implements Document {
    public void open() {
        System.out.println("Opening Word Document");
    }
}

class PdfDocument implements Document {
    public void open() {
        System.out.println("Opening PDF Document");
    }
}

class ExcelDocument implements Document {
    public void open() {
        System.out.println("Opening Excel Document");
    }
}

abstract class DocumentFactory {
    public abstract Document createDocument();
}

class WordFactory extends DocumentFactory {
    public Document createDocument() {
        return new WordDocument();
    }
}

class PdfFactory extends DocumentFactory {
    public Document createDocument() {
        return new PdfDocument();
    }
}

class ExcelFactory extends DocumentFactory {
    public Document createDocument() {
        return new ExcelDocument();
    }
}

public class FactoryMethodPatternExample {

    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordFactory();
        Document word = wordFactory.createDocument();
        word.open();

        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
    }
}
```

```

        pdf.open();

        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}

```



Module 2

Exercise 2: E-commerce Platform Search Function

```

import java.util.Scanner;

class Product {

    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
}

class LinearSearch {

    public static Product search(Product[] products, int id) {

        for (int i = 0; i < products.length; i++) {

            if (products[i].productId == id) {
                return products[i];
            }

        }

        return null;
    }
}

class BinarySearch {

```

```

public static Product search(Product[] products, int id) {

    int low = 0;
    int high = products.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (products[mid].productId == id) {
            return products[mid];
        } else if (products[mid].productId < id) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return null;
}

}

public class EcommerceSearch {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Product[] productsLinear = {
            new Product(103, "Shoes", "Fashion"),
            new Product(101, "Laptop", "Electronics"),
            new Product(105, "Bag", "Fashion"),
            new Product(102, "Phone", "Electronics"),
            new Product(104, "Watch", "Accessories")
        };

        Product[] productsBinary = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Phone", "Electronics"),
            new Product(103, "Shoes", "Fashion"),
            new Product(104, "Watch", "Accessories"),
            new Product(105, "Bag", "Fashion")
        };

        System.out.print("Enter Product ID to search: ");
        int searchId = sc.nextInt();

        Product linearResult = LinearSearch.search(productsLinear, searchId);

        if (linearResult != null) {
            System.out.println("\nLinear Search Result:");
            System.out.println("Product ID: " + linearResult.productId);
            System.out.println("Product Name: " + linearResult.productName);
            System.out.println("Category: " + linearResult.category);
        } else {
            System.out.println("\nProduct not found using Linear Search");
        }
    }
}

```

```

        Product binaryResult = BinarySearch.search(productsBinary, searchId);

        if (binaryResult != null) {
            System.out.println("\nBinary Search Result:");
            System.out.println("Product ID: " + binaryResult.productId);
            System.out.println("Product Name: " + binaryResult.productName);
            System.out.println("Category: " + binaryResult.category);
        } else {
            System.out.println("\nProduct not found using Binary Search");
        }

        sc.close();
    }
}

```

```

Run EcommerceSearch
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Users\DELL\AppData\Local\Programs\IntelliJ IDEA Community Edition 2025.2\lib\idea_rt...
Enter Product ID to search: 182

Linear Search Result:
Product ID: 182
Product Name: Phone
Category: Electronics

Binary Search Result:
Product ID: 182
Product Name: Phone
Category: Electronics

Process finished with exit code 0

```

Big O Notation

Big O notation is used to measure the efficiency of an algorithm by describing how its execution time grows as the input size increases.

Linear Search:

- Best Case: $O(1)$
- Average Case: $O(n)$
- Worst Case: $O(n)$

Binary Search:

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Binary Search is more suitable for an e-commerce platform because it searches much faster by repeatedly dividing the search space into halves. However, the product array must be sorted before using Binary Search. For large product databases, Binary Search provides better performance than Linear Search.

Exercise 7: Financial Forecasting

Recursion is a technique where a method calls itself to solve a problem by breaking it into smaller subproblems.

Example

Factorial:

5!
= $5 \times 4!$
= $5 \times 4 \times 3!$
= ...
= 120

Here, the function repeatedly calls itself until it reaches the base case.

Advantages

- Simple code
- Easy to solve repetitive problems
- Useful for tree traversal, searching, forecasting, factorial, Fibonacci, etc.

```
public class FinancialForecast {  
  
    static double forecast(double currentValue,  
                           double growthRate,  
                           int years) {  
  
        if (years == 0)  
            return currentValue;  
  
        return forecast(currentValue,  
                        growthRate,  
                        years - 1)  
            * (1 + growthRate);  
    }  
  
    public static void main(String[] args) {  
  
        double currentValue = 10000;  
        double growthRate = 0.10; //10%  
        int years = 5;  
  
        double futureValue =  
            forecast(currentValue,  
                    growthRate,  
                    years);  
  
        System.out.println("Future Value = " + futureValue);  
    }  
}
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\DELL\AppData\Local\Programs\IntelliJ IDEA Community Edition 2025.2\lib\ide-  
Future Value = 16185.188688000000  
  
Process finished with exit code 0
```

Time Complexity

Each recursive call processes one year.

If there are **n** years:

Time Complexity = $O(n)$

because the method makes **n** recursive calls.

Space Complexity:

$O(n)$

because each recursive call is stored on the call stack.

Use Iteration to make it optimize

double future = currentValue;

```
for(int i=1;i<=years;i++)  
{  
    future = future * (1 + growthRate);  
}
```

Module 3

Exercise 1: Control Structures

Scenario 1:

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

```
CREATE TABLE Customer (
```

```
    CustomerID INT PRIMARY KEY,
```

```
    CustomerName VARCHAR(50),
```

```
    Age INT,
```

```
    Balance DECIMAL(10,2),
```

```
    IsVIP BOOLEAN
```

```
);
```

```
CREATE TABLE Loan (
```

```
    LoanID INT PRIMARY KEY,
```

```
    CustomerID INT,
```

```
    InterestRate DECIMAL(5,2),
```

```
    DueDate DATE
```

```
);
```

```
INSERT INTO Customer VALUES
```

```
(101,'Akash',65,15000,0),
```

```
(102,'Riya',45,8000,0),
```

```
(103,'Rahul',70,12000,0),
```

```
(104,'Sneha',30,5000,0);
```



```

INSERT INTO Loan VALUES
(1,101,8.50,CURDATE()+INTERVAL 10 DAY),
(2,102,9.00,CURDATE()+INTERVAL 45 DAY),
(3,103,7.50,CURDATE()+INTERVAL 20 DAY),
(4,104,8.00,CURDATE()+INTERVAL 15 DAY);

DELIMITER $$

DROP PROCEDURE IF EXISTS DiscountInterest;

DELIMITER $$

CREATE PROCEDURE DiscountInterest()

BEGIN

    UPDATE Loan L

    JOIN Customer C

    ON L.CustomerID = C.CustomerID

    SET L.InterestRate = L.InterestRate - 1

    WHERE C.Age > 60;

    SELECT 'Interest Discount Applied Successfully' AS Message;

END $$

DELIMITER ;

CALL DiscountInterest();

```

Scenario 2:

```

DROP PROCEDURE IF EXISTS UpdateVIP;

DELIMITER $$

```

```

CREATE PROCEDURE UpdateVIP()

BEGIN

    UPDATE Customer

    SET IsVIP = TRUE

    WHERE Balance > 10000;

    SELECT 'VIP Status Updated Successfully' AS Message;

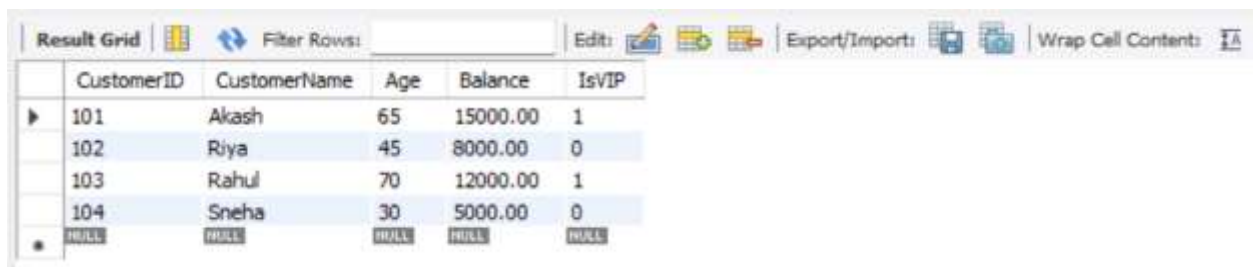
END $$

DELIMITER ;

CALL UpdateVIP();

SELECT * FROM Customer;

```



	CustomerID	CustomerName	Age	Balance	IsVIP
▶	101	Akash	65	15000.00	1
	102	Riya	45	8000.00	0
	103	Rahul	70	12000.00	1
	104	Sneha	30	5000.00	0
*	NULL	NULL	NULL	NULL	NULL

Scenario 3:

```

DROP PROCEDURE IF EXISTS LoanReminder;

DELIMITER $$

CREATE PROCEDURE LoanReminder()

BEGIN

    SELECT

        C.CustomerID,

        C.CustomerName,

        L.LoanID,

```

```

L.DueDate,

CONCAT('Reminder: Dear ', C.CustomerName,

      ', your loan is due on ', L.DueDate) AS Reminder

FROM Customer C

INNER JOIN Loan L

ON C.CustomerID = L.CustomerID

WHERE L.DueDate BETWEEN CURDATE()

      AND DATE_ADD(CURDATE(), INTERVAL 30 DAY);

END $$

DELIMITER ;

CALL LoanReminder();

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	CustomerID	CustomerName	LoanID	DueDate	Reminder
▶	101	Akash	1	2026-07-05	Reminder: Dear Akash, your loan is due on 202...
	103	Rahul	3	2026-07-15	Reminder: Dear Rahul, your loan is due on 2026...
	104	Sneha	4	2026-07-10	Reminder: Dear Sneha, your loan is due on 202...

Exercise 3: Stored Procedures

Scenario 1:

```

CREATE TABLE SavingsAccount (

    AccountID INT PRIMARY KEY,

    CustomerName VARCHAR(50),

    Balance DECIMAL(10,2)

);

INSERT INTO SavingsAccount VALUES

```

```

(101,'Akash',10000),
(102,'Riya',25000),
(103,'Rahul',15000);

DROP PROCEDURE IF EXISTS ProcessMonthlyInterest;

DELIMITER $$

CREATE PROCEDURE ProcessMonthlyInterest()

BEGIN

    UPDATE SavingsAccount

    SET Balance = Balance + (Balance * 0.01);

    SELECT 'Monthly Interest Applied Successfully' AS Message;

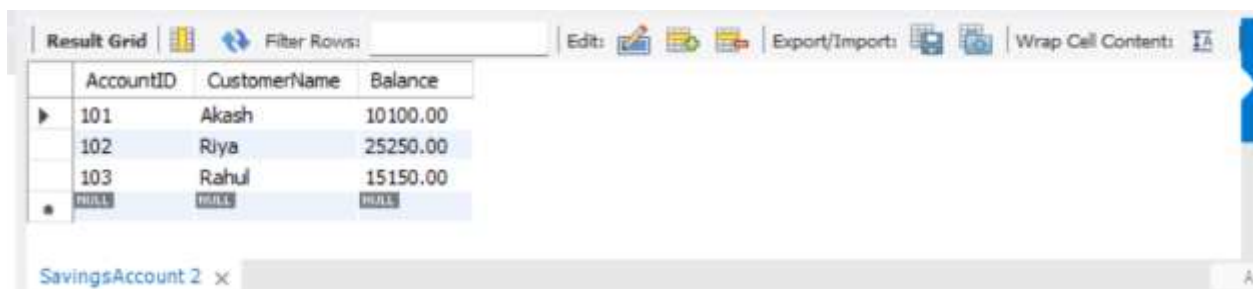
END $$

DELIMITER ;

CALL ProcessMonthlyInterest();

SELECT * FROM SavingsAccount;

```



	AccountID	CustomerName	Balance
▶	101	Akash	10100.00
	102	Riya	25250.00
	103	Rahul	15150.00
•	NULL	NULL	NULL

SavingsAccount 2 x

Scenario:2

```

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    Department VARCHAR(30),

```


Scenerio 3:

```
CREATE TABLE Account (  
    AccountID INT PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    Balance DECIMAL(10,2)  
);  
  
INSERT INTO Account VALUES  
(101,'Akash',20000),  
(102,'Riya',10000);  
  
DROP PROCEDURE IF EXISTS TransferFunds;
```

DELIMITER \$\$

```
CREATE PROCEDURE TransferFunds
```

```
(  
    IN FromAcc INT,  
    IN ToAcc INT,  
    IN Amount DECIMAL(10,2)  
)
```

```
BEGIN
```

```
    DECLARE SourceBalance DECIMAL(10,2);
```

```
    SELECT Balance
```

```
    INTO SourceBalance
```

```
    FROM Account
```

```
    WHERE AccountID = FromAcc;
```

```
    IF SourceBalance >= Amount THEN
```

```
        UPDATE Account
```

```
        SET Balance = Balance - Amount
```

```
WHERE AccountID = FromAcc;

UPDATE Account

SET Balance = Balance + Amount

WHERE AccountID = ToAcc;

SELECT 'Transfer Successful' AS Message;

ELSE

    SELECT 'Insufficient Balance' AS Message;

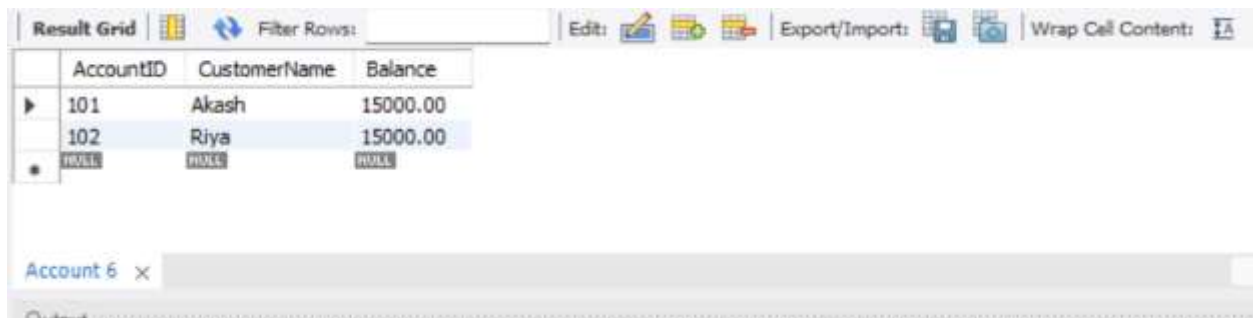
END IF;

END $$

DELIMITER ;

CALL TransferFunds(101,102,5000);

SELECT * FROM Account;
```



The screenshot shows a database client interface with a toolbar at the top containing icons for 'Result Grid', 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. Below the toolbar is a table with the following data:

	AccountID	CustomerName	Balance
▶	101	Akash	15000.00
	102	Riya	15000.00
•	NULL	NULL	NULL

At the bottom of the window, there is a tab labeled 'Account 6' with a close button (X).